Interactive Object Tracking on Gait-Parameterized Locomotion Policy

Gia-Hung Huynh[1], Tianxin Hu[2], Tien-Dat Nguyen[1,3],
Shenghai Yuan[2], *Member, IEEE*, Thien-Minh Nguyen[1†], *Member, IEEE*

***Abstract*— Gait-parameterized locomotion policies have been recognised as an effective control for adapting legged robots' locomotion behaviour across different terrains. In this paper, we investigate how such locomotion policies can be leveraged for interactive tasks beyond pure locomotion. Rather than training a task-specific policy in an end-to-end manner, we treat the trained policy as an inner-loop controller and exploit gait parameters as control inputs within a larger feedback-control framework. To this end, we first realise a sim-to-real deployment of a parameterized quadrupedal locomotion policy trained using the Isaac Lab / NVIDIA Isaac Sim reinforcement learning framework. The resulting controller supports multiple agile locomotion behaviours, including trotting, pronking, pacing, and bounding gaits, with controllable footfall frequencies, body heights, and body attitudes. Building upon this locomotion interface, we implement a vision-based ball detection and tracking system coupled with a lightweight visual feedback controller. Experimental results demonstrate real-time object tracking capability by interactively executing different *gait expressions* during task execution, which informs humans of the internal operational state. This study further illustrates how reinforcement-learned locomotion policies can be effectively integrated with lightweight classical feedback control to realise interactive quadrupedal robotic behaviours.**

I. Introduction

In recent years, legged robots have emerged as a major platform for robot learning, driven by rapid reductions in hardware cost and the development of GPU-accelerated simulation and reinforcement learning frameworks [1]–[3]. In tandem with these developments, learned locomotion policies have reached a high degree of maturity, enabling quadrupedal robots to traverse complex terrains with agile and adaptive behaviours [4]–[8]. Inspired by the innate locomotion capabilities of humans and animals, these learned policies can operate purely from proprioceptive inputs, namely the robot's internal states such as IMU measurements and joint positions and velocities, without requiring complex state estimation from exteroceptive sensors such as cameras and LiDAR [9], [10]. Other notable advances include the unified quadrupedal and bipedal locomotion policy reported in [11], as well as platform-agnostic locomotion policies demonstrated in [12].

Of particular interest to our work is a class of gait-parameterized locomotion policies, as pioneered in [8]. In this work, the authors developed a velocity-tracking policy featuring distinct locomotive *gaits* inspired by animal locomotion patterns, including *trotting*, *pronking*, *pacing*, and *bounding*. More importantly, the policy exposes a set of interpretable locomotion parameters, including foot contact phases, frequency, body height, pitch, and roll, allowing a single learned policy to generate a diverse range of locomotion behaviours. Indeed, the natural emergence of the *pronking* gait from elegant in-phase foot-contact coordination is particularly interesting, as it enables jumping-like behaviours that traditionally require either dedicated complex training curricula [13] or computationally intensive model-predictive sampling strategies [14].

Fig. 1: The robot platform (left), and illustration of the object tracking result and the applied tracking rule from Tab. III.

The original motivation behind such parameterized policies was to allow users or higher-level planners to directly adjust locomotion behaviour according to terrain conditions. Different from this original intention, we posit that by cascading a gait-parameterized locomotion policy with other controllers, it becomes possible to achieve more complex tasks beyond locomotion itself. Such a controller may take the form of another policy interfacing with the gait-parameterized policy, leveraging lightweight perception and feedback-control modules to infer appropriate gait control signals for the underlying locomotion controller. Different from other approaches that train the policies for the task-specific policies like badminton [15], self-balancing [16], [17], or RL-augmented model-based control [18], this hierarchical architecture may provide a practical pathway toward interactive robotic behaviours, both with the surrounding environment and human operators.

As a preliminary study, this paper seeks to realise such a multi-layer control framework with simple classic controller in the outer loop. In particular, rather than introducing

[1] School of Mechanical and Mining Engineering, The University of Queensland, St Lucia, QLD 4067
[2] School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore 639798, 50 Nanyang Avenue
[3] Faculty of Electrical and Electronic Engineering, Ho Chi Minh City University of Technology (HCMUT), VNU-HCM
[†] Corresponding author. E-mail: thienminh.nguyen@uq.edu.au

another learned policy, we employ a lightweight classical controller based on simple control and decision rules, coupled with a lightweight object detection module that provides visual feedback for closed-loop control.

The contributions of this paper are summarised as follows:

- We develop the reinforcement learning pipeline for a gait-parameterized quadrupedal locomotion policy using the latest Isaac Sim and Isaac Lab training framework. The process achieves successful sim-to-real transfer on the Unitree Go2 quadrupedal model with significantly fewer iterations, and requires only a single stage to train both policy and estimator.
- We demonstrate the feasibility of integrating the gait-parameterized locomotion policy as an inner-loop controller, while employing a classical feedback controller in the outer loop to achieve interactive robotic behaviours beyond locomotion.

The remainder of the paper is organized as follows. Sec. II gives an overview of the system, including the problem formulation. Sec. III reports the details of the multi-layer object tracking pipeline, consisting of our formulation of the gait-parameterized locomotion policy in Sec. III-A, the training techniques to achieve successful sim-to-real deployment on real robot in Sec. III-B, and the interactive perception and control loop for object tracking Sec. III-C. We report on the experiments with the complete system and evaluate the results in Sec. IV. Finally, Sec. V concludes our work.

II. System Overview

This Section will explain the system as illustrated in Fig. 2. The system is realised in two stages. First, we attempt to realise a gait-parameterized locomotion policy $\pi_g(c, o)$ using a reinforcement learning pipeline. The training is done on a simulation, which follows a curriculum to gradually train the robot to track the velocity command at increasingly larger speed, with randomly sampled gait parameters. In addition, domain randomization is employed to ensure successful real-world deployment. Once the policy is verified by manually controlling the robot using a command from the Xbox controller, we can then switch the command to the ones generated from the tracking policy $\pi_{\text{track}}(b)(\cdot)$. We explain some of the main components of this system below.

Gait-parameterized policy: The policy $\pi_g(c, o)$ is at the centre of our system. It takes in a command c and an observation o (which will be elaborated later), and outputs an action $a \in \mathbb{R}^{12}$. The action a is indeed the setpoints q^* of the hip, thigh and calf joints on the four legs. These joint setpoints will be trimmed prior to being sent over the API to be executed by the internal PD controller.

Command: We denote $c = (v^*_{xy}, \omega^*_z, g, \theta)$ as the gait-parameterized command. More specifically, $v^*_{xy} \in \mathbb{R}^2$ is the desired linear velocity, $\omega^*_z \in \mathbb{R}$ is the desired yaw rate, $g \in \{0, 1, 2, 3\}$ denotes the gait choice, corresponding to the trotting, pronking, pacing, bounding gaits, and $\theta \in \mathbb{R}^N$ encodes other gait parameters. Specifically, θ describes the footfall frequency, body height, roll, pitch, stance width and length while executing one of the gaits. More details of θ will be provided in Sec. III-A

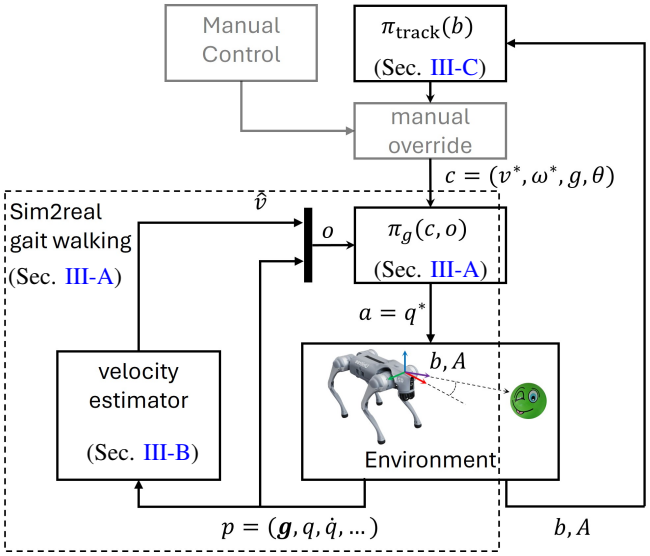


Fig. 2: Overview of the system at the final deployment stage. The "gait walking" subsystem consists of the learned policy $\pi_g(\cdot)$, the simulated or real-world environment, and a velocity estimator. The policy $\pi_g(\cdot)$ takes in the command c and observation o, and generates the action a in the form of the desired joint position q^*, enabling trotting, pronking, pacing, and bounding gaits while tracking the commanded velocity v^*_{xy}. Building upon this manual velocity tracking and gait control capability, we implement an object tracking control law that generates the corresponding command c based on the relative target bearing b. Manual control can also optionally override the autonomous tracking command. More details are provided in Sec. II and Sec. III.

Observation: The observation input to the policy $\pi_g(\cdot)$ is denoted as $o = (\hat{v}_{xy}, p, \ldots)$, where $p = (\boldsymbol{g}, q, \dot{q})$ is the proprioceptive observation, consisting of the IMU observation $i \in \mathbb{R}^3$ (a unit-vector reflecting the coordinate of gravity in body frame), and $q, \dot{q} \in \mathbb{R}^{12}$ are respectively the positions and velocities of the 12 joints on the robot's legs. Besides these, we also include the clock inputs for each foot, the current and past actions. This proprioception p will feed into the estimator to produce the velocity estimate $\hat{v}_{xy} \in \mathbb{R}^2$.

Object tracking controller: On top of the gait-parameterized policy is the object tracking policy $\pi_{\text{track}}(\cdot)$. This policy is a mix of logic-based and proportional controllers to generate the command c based on the object-tracking information b, comprising of the bearing, or an indication of the object being out-of-FOV. Currently, we only consider a simple object that can be detected by basic machine vision techniques.

More details on the realisation of these policies are provided in Sec. III.

III. Methodology

A. Gait-parameterized Policy Development

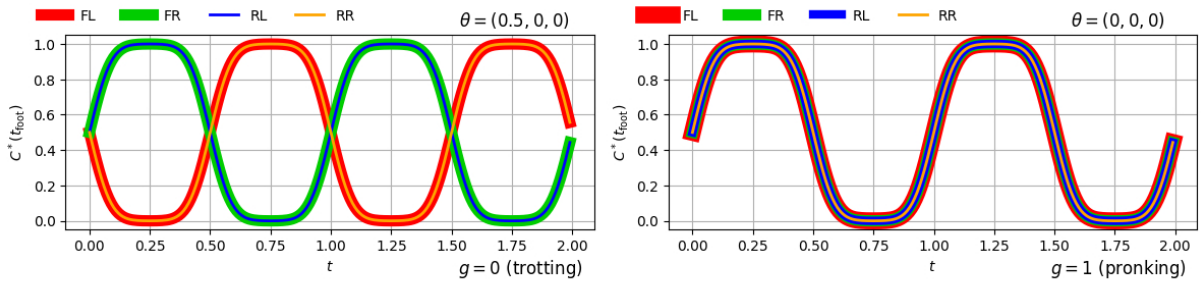


Fig. 3: The desired contact state of the feet for the *trotting* and *pronking* gaits as realized by equations (1), (2), (3). Notice the in-phase contact states of the pairs of feet of the different gaits (FL, FR, RR, RL refer to front-left/right and rear-left/right limb).

TABLE I: Reward structure of the gait-parameterized locomotion policy training and the range of each parameter.

Category	Term	Equation	Gait Parameter Range	Weight
Task	Linear velocity tracking	$\exp\left(-\|v_{xy} - v^*_{xy}\|^2/0.25\right)$	$v^*_{xy} \in [-1.0, 1.0] \times [-1.0, 1.0]$	100
Task	Angular velocity tracking	$\exp\left(-\|\omega_z - \omega^*_z\|^2/0.25\right)$	$\omega^*_z \in [-1.0, 1.0]$	60
GL	Swing phase force tracking	$\sum_{\text{foot}} \frac{1}{4}\left[1 - C^*(\sigma, t_{\text{foot}})\right] \exp\left(-\|\mathbf{f}_{\text{foot}}\|^2/100\right)$	$\sigma = 0.07$	-3
GL	Stance phase velocity tracking	$\sum_{\text{foot}} \frac{1}{4} C^*(\sigma, t_{\text{foot}}) \exp\left(-\|v_{\text{foot},xy}\|^2/10\right)$	$\sigma = 0.07$	-3
GL	Body height tracking	$(h_z - h^*_z)^2$	$h^*_z \in [-0.25, 0.25]$	-10
GL	Body roll-pitch tracking	$\left\|\boldsymbol{g}_{xy} - \boldsymbol{g}^*_{xy}\right\|^2$	pitch, roll $\in [-0.4, 0.4] \times [-0.2, 0.2]$	-7.5
GL	Foot-swing tracking	$\left\|\boldsymbol{p}_{\text{foot},xy} - \boldsymbol{p}^*_{\text{foot},xy}\right\|^2$	stance width / length $\in [0.2, 0.4] \times [0.3, 0.5]$	-5.0
GL	Foot-swing height tracking	$\sum_{\text{foot}} (h_{\text{foot},z} - h^*_{\text{foot},z})^2 \left[1 - C^*(\sigma, t_{\text{foot}})\right]$	$\sigma = 0.07$, $h^*_{\text{foot},z} = 0.1$	-7.5
Stability	z velocity	v_z^2	–	-0.01
Stability	Roll-pitch velocity	$\|\omega_{xy}\|^2$	–	-0.01
Stability	Foot slip	$\sum_{\text{foot}} \|v_{\text{foot},xy}\|^2 \mathbf{c}_{\text{foot}}$	–	-10
Stability	Thigh/calf collision	$\sum_{\text{thigh}} \mathbb{1}(\|\mathbf{f}_{\text{thigh}}\| > 0.1) + \sum_{\text{calf}} \mathbb{1}(\|\mathbf{f}_{\text{calf}}\| > 0.1)$	–	-5
Reg.	Action smoothing	$\|\mathbf{a}_{\text{current}} - \mathbf{a}_{\text{previous}}\|^2$	–	-0.01

Note: GL and Reg. refer to Gait Learning and Regularization, respectively; $\mathbf{f}_{\text{foot}}$ denotes the net contact force acting on foot foot; h_z and h^*_z are the current and commanded body height; $h_{\text{foot},z}$ and $h^*_{\text{foot},z}$ are the current and desired height of foot foot; $\boldsymbol{g}_{xy}$ and $\boldsymbol{g}^*_{xy}$ denote the measured and commanded xy gravity vector in the robot body frame; $\boldsymbol{p}_{\text{foot},xy}$ and $\boldsymbol{p}^*_{\text{foot},xy}$ are the current and desired xy foot position; $\mathbf{c}_{\text{foot}} = 1$ if foot experienced collision during the previous control interval; $\mathbb{1}(\cdot)$ is an indicator function returning 1 when the enclosed condition is satisfied. The commanded quantities $(^*)$ are determined by the gait parameter vector θ.

1) Gait Parameterization Formulation: Inspired by [8], we define the contact state function for each foot as follows:

$$C^*(t_{\text{foot}}) = \Phi(\sigma, t_{\text{foot}})\left[1 - \Phi(\sigma, t_{\text{foot}} - 0.5)\right] + \Phi(\sigma, t_{\text{foot}} - 1)\left[1 - \Phi(\sigma, t_{\text{foot}} - 1.5)\right], \quad (1)$$

where $\Phi(\sigma, t) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x}{\sigma}\right)^2}$, $t_{\text{foot}} \in [0, 1]$, foot $\in \{\text{FL}, \text{FR}, \text{RL}, \text{RR}\}$, and σ is a user-defined parameter.

Given a time reference t, and an updated frequency f, we define $\bar{t} = t/\text{f}$ as the common clock signal. By modifying the phase of t_{foot} relative to the clock reference $\bar{t}$ for each foot, different gaits will manifest. Specifically, we can describe the footfall pattern of the gaits using the following equations

$$t_{\text{FL}} = (\bar{t} + \phi_1 + \phi_2 + \phi_3)\%1, \;\; t_{\text{FR}} = (\bar{t} + \phi_2)\%1, \\ t_{\text{RL}} = (\bar{t} + \phi_3)\%1, \;\; t_{\text{RR}} = (\bar{t} + \phi_1)\%1, \;\; \bar{t} = t/\text{f}, \quad (2)$$

where $\%$ returns the float-valued remainder of division. The phase parameter $\phi \triangleq (\phi_1, \phi_2, \phi_3)$ is set based on the gait choice g as follows:

$$\phi = (\phi_1, \phi_2, \phi_3) = \begin{cases} (0.5,\ 0,\ 0), & g = 0 \\ (0,\ 0,\ 0), & g = 1 \\ (0,\ 0.5,\ 0), & g = 2 \\ (0,\ 0,\ 0.5), & g = 3 \end{cases}, \quad (3)$$

where $g = (0, 1, 2, 3)$ corresponds to the the *trotting*, *pronking*, *pacing*, and *bounding* gaits, respectively. Fig. 3 illustrates the contact schedule pattern of the two main gaits.

Here we fix the value of the phase tuple ϕ for each gait according to the gait choice g, and include g as part of the observation input to the policy. This differs from [8], which does not formulate the gait choice g, and instead samples each phase parameter ϕ_i from the interval $[0.25, 0.75]$ to achieve a multiplicity of behaviours. While the original gait parameterization over a continuous space theoretically allows a larger range of gaits, in practice, we find that this may slow down training convergence, and the resulting locomotion behaviours often collapse toward the nominal gaits corresponding to the four values of ϕ listed in (3).

2) Rewards Design: Tab. I reports the reward functions used to train the robot in the simulation. We define a time

TABLE II: Domain randomization events during training in simulation at three stages (**Stg.**): S – startup (once at initialization), R – reset (at every episode reset), and I – interval (periodically during execution).

Stg.	Random Event	Parameter Range
S	Rigid body material	Static friction $\in [0.05, 1.5]$, dynamic friction $\in [0.05, 1.2]$, restitution $\in [0, 0.5]$.
S	Base mass & inertia	Scale factor $\in [0.85, 1.15]$.
S	Base center of mass	$x \in [-0.015, 0.015]$, $y \in [-0.01, 0.01]$, $z \in [-0.01, 0.01]$.
S	Joint friction & armature	Friction $\in [0, 0.3]$, armature $\in [0, 0.03]$.
R	Joint initialization	Joint position offset $\in [-0.75, 0.75]$, joint velocity $\in [-1.5, 1.5]$.
R	Actuator gains	Stiffness scale $\in [0.7, 1.3]$, damping scale $\in [0.7, 1.3]$ (log-uniform).
R	Joint damping	Damping $\in [0.05, 0.15]$
R	Ext. force/torque on base	Force $\in [-8, 8]$ N, torque $\in [-2, 2]$ Nm.
R	Root state initialization	Height $z \in [0.24, 0.36]$, yaw $\in [-0.2, 0.2]$, linear/angular velocity $\in [-0.75, 0.75]$.
I	Velocity perturbation (push)	Linear velocity disturbance in $x, y, z \in [-0.5, 0.5]$ every 2–10 s.
I	Gravity perturb.	Gravity offset sampled from Gaussian distribution with bounds $x, y \in [-0.25, 0.25]$, $z \in [-0.4, 0.4]$ every 36 s.

TABLE III: Object tracking control policy $\pi_{\text{track}}(b)$.

#	Condition	Command (v^*_{xy}, ω^*, g)
(1)	$\mathbf{d} \wedge (\lvert\psi\rvert > 30°) \wedge (A < A_M)$	$\big((0,0), \text{clip}(-k_\omega \psi, \omega_M), 0\big)$
(2)	$\mathbf{d} \wedge (\lvert\psi\rvert \leq 30°) \wedge (A < A_M)$	$\big((\text{clip}(k_v(A^* - A), v_M), 0), 0, 0\big)$
(3)	$\mathbf{d} \wedge (\lvert\psi\rvert > 30°) \wedge (A \geq A_M)$	$\big((0,0), \text{clip}(-k_\omega \psi, \omega_M), 0\big)$
(4)	$\mathbf{d} \wedge (\lvert\psi\rvert \leq 30°) \wedge (A \geq A_M)$	$\big((0,0), 0, 1\big)$
(5)	Otherwise	$\big((0,0), \omega_S, 1\big)$

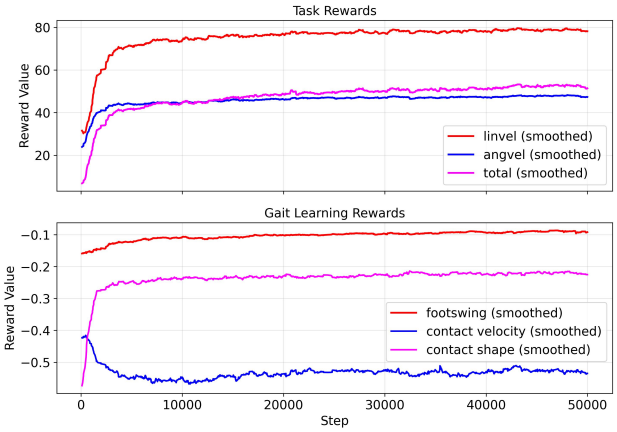


Fig. 4: Main rewards of the task and gait learning terms in Tab. I. Training reaches saturation past 40000 steps.

interval of $20ms$ to update the robot states, calculate the rewards, and update the policy.

B. Sim-to-real Transfer Techniques

Domain randomization is important to ensure the trained policy is robust to the uncertainty in the actual physical robot at deployment. Tab. II reports the ranges of the simulated parameters in our simulation.

State observer while velocity is included as an observation for the policy, this measurement is often only available with the use of onboard localization and perception systems such as visual-inertial odometry (VIO), lidar-inertial odometry (LIO). Following the approaches in literature, we develop a state observer to estimate the lateral velocity, using similar observations of the actor-critic network.

Network architecture Inspired by [10], instead of using the MLP for the policy networks and estimator as in [8], we adopt the LSTM architecture for the neural networks used in this work. We notice that this architecture allows for a more accurate state estimation and a reliable policy. The actor and critic networks both have 3 hidden layers with dimensions 512, 256, 128. The state estimator has 2 hidden layers with dimensions 256 and 128.

IssacLab Settings: Training was done using the PPO process following standard parameters recommended by IsaacLab. Different from [8] that required 2.58×10^9 iterations to convergence, we find that only 50×10^3 iterations are required to achieve convergence. This may be due to the significantly reduced sampling space for v_{xy} from up to 5m/s to 1m/s (see Tab. I), and the finite set of values for the parameter ϕ. In addition, we only sample the linear velocity command v^*_{xy} in the $[-1, 1] \times [-1, 1]$ range.

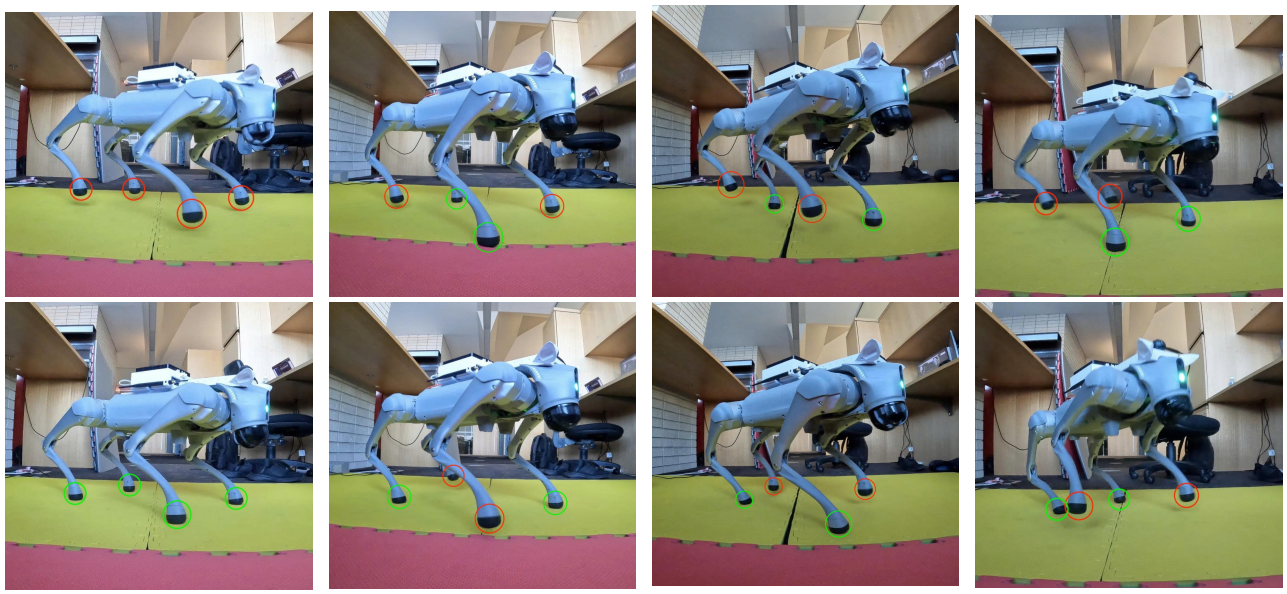
The standard procedure in previous works requires a two-stage training procedure. In the first stage, we need to train both the actor and the estimator networks with ground truth velocity v_{xy}. In the second stage, the estimator's velocity estimate should replace the true velocity in the observation input of $\pi_g(\cdot)$ as in Fig. 2. However, we find that it is sufficient to train the policy network and the estimator in one stage, and the whole training period only requires approximately 12 hours.

Deployment: The gait-parameterized policy is deployed on a Unitree Go2 robot, with onboard computation provided by a Jetson Orin NX (Fig. 1). The deployment software stack is implemented in ROS 2 Humble and consists of several ROS nodes. These nodes collect the proprioceptive information, the control inputs from an Xbox controller, convert them to the gait parameters at the appropriate scale, and assemble them into the appropriate observation tensor for the policy. A separate thread periodically performs policy inference to generate the action output, which is then sent to the robot's onboard computer for execution as the desired joint target command. The inference update rate is fixed at 50 Hz, matching the original training setup.

C. Interactive Perception and Tracking

Given the bearing observation b, we first compute the relative yaw angle as $\psi = \text{atan2}(b_y, b_x)$. We define the clipping function as $\text{clip}(a, b) \triangleq \frac{b}{\max(\|a\|, b)} a$, and let the indicator $\mathbf{d}$ return 1 when the object is detected, and 0 otherwise. The object-tracking controller $\pi_{\text{track}}(b)$ generates the command tuple (v^*_{xy}, ω^*, g) according to Tab. III.

In essence, when the object is detected, a clipped rotational command is first generated to reduce the bearing error below



(a) Pronk (b) Trot (c) Pace (d) Bound

Fig. 5: Achieved gaits on the real Unitree Go2. Each column shows one gait at phase A (top) and phase B (bottom); Green circles mark feet in ground contact (stance); Red circles mark feet in the air (swing). **(a)** Pronk: all four feet in phase; **(b)** Trot: diagonal pairs alternate; **(c)** Pace: lateral pairs move together; **(d)** Bound: front and hind pairs alternate.

a predefined threshold. Once aligned, the robot transitions to a forward motion command with clipped linear velocity, proportional to the observed size of the target in the image. The robot will pronk in place to signal that the target is within the threshold A^8.

When the object is no longer detected, the controller switches to an in-place rotational search motion while adopting the pronking gait. This change in gait expression provides a visual cue to the human operator that tracking has been lost, allowing manual intervention if needed. This interaction mechanism motivates the term *interactive object tracking*.

IV. Results & Evaluation

A recording of the gait behaviours and the object-tracking demonstration is available at https://youtu.be/IC0k1u7NGM8.

A. Gait Behaviour

Fig. 4 shows the rewards of the in-simulation training. After sim-to-real transfer, the learned policy $\pi_g(\cdot)$ reliably reproduces all four commanded gaits: trotting, pronking, pacing and bounding on the physical Unitree Go2 under teleoperated velocity commands. Fig. 5 shows each gait at two phases of its cycle, with feet in stance (ground contact) and swing (air) marked in green and red, respectively. The characteristic contact signatures defined by (1)–(3) are recovered on hardware: all four feet move in phase for the *pronk*, diagonal pairs alternate for the *trot*, lateral pairs move together for the *pace*, and the front and hind pairs alternate for the *bound*, matching the contact schedules designed in simulation (Fig. 3).

B. Object Tracking

We evaluate the complete tracking system on the real robot using the controller of Tab. III, with a green ball as the target and the bearing b and apparent target area A obtained from the onboard camera alone. No external localization or manual velocity command is used while the robot is tracking autonomously. Fig. 1 provides snapshots of the tracking process from the robot's front view.

Fig. 6 (left) logs a representative run, with $t = 0$ set at the first autonomous command. For $t \in (0, 9)$ we observe the robot repeatedly advancing toward the ball (cond. 2). When the target becomes sufficiently large, the robot executes a fixed in-place pronk for about 3s as an arrival cue (cond. 4) before the operator moves the ball further ahead, demonstrating an approach–pronk–approach cycle, as shown in the top left of Fig. 1. Whenever the bearing observation leaves the $[-30°, 30°]$ range (around the 9s–12s and 16s–19s intervals), forward motion is temporarily replaced by the clipped yaw command until the target is re-aligned (cond. 1). In the 61s–73s interval, the operator removes the ball from view, causing detection to drop and the robot to switch to an in-place pronking search (cond. 5) for about 12s until it re-acquires the target and resumes tracking. The operator briefly overrides the autonomous control around seconds 89 to 92, confirming that manual commands can pre-empt the autonomous policy at any time.

The same run also demonstrates the interactive aspect of the system. When the operator moves the ball out of the field of view, the robot loses the target and switches to an in-place pronking search, which both scans the surroundings and provides a clear visual cue that tracking has been lost. Once the ball is brought back into view, the robot re-acquires

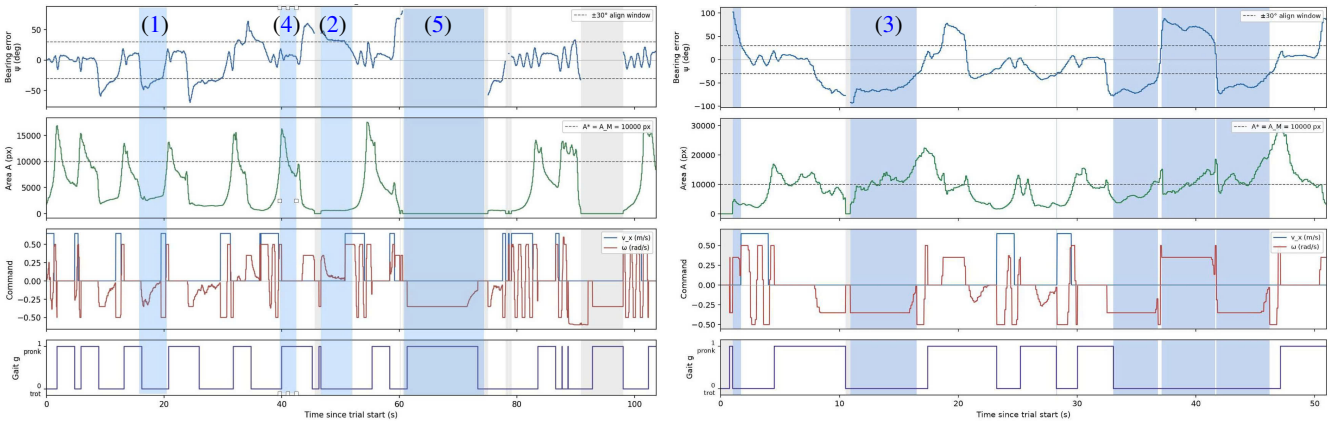


Fig. 6: Closed-loop object-tracking telemetry from two representative real-robot runs, recorded from onboard sensing alone. Each plot stacks four signals: **(i)** the bearing error $\psi = \operatorname{atan2}(b_y, b_x)$ with the dashed $\pm 30^\circ$ alignment window; **(ii)** the target area A with the dashed stopping threshold $A^* = A_M = 10^4$ px; **(iii)** the issued command, forward velocity v_x and yaw rate ω; and **(iv)** the commanded gait g (trot vs. pronk). Numbered tags mark the active row of Tab. III; grey spans denote "object not detected" (search), and blue spans highlight representative align-then-approach episodes.

it and resumes tracking, allowing the operator to redirect the robot simply by repositioning the target. Upon reaching the ball, the robot again adopts a brief pronking motion to signal arrival. Together, these behaviours realise the intended interaction loop, in which distinct gait expressions communicate the controller's internal state to a human observer.

V. Conclusion & Future works

In this paper, we presented a multi-layer control framework that leverages a gait-parameterized locomotion policy as an inner-loop controller for interactive robotic behaviours beyond locomotion itself. We developed a reinforcement learning pipeline in Isaac Lab to realise a quadrupedal policy capable of executing multiple agile gaits, and demonstrated successful sim-to-real deployment on the Unitree Go2 platform. Building upon this locomotion capability, we implemented a lightweight vision-based object tracking controller that generates gait commands from visual feedback, enabling real-time interactive object tracking. The results demonstrate the feasibility of integrating reinforcement-learned locomotion policies with lightweight classical control to realise more complex robotic behaviours.

Future works would include a

References

[1] V. Makoviychuk, L. Wawrzyniak, Y. Guo *et al.*, "Isaac gym: High performance gpu-based physics simulation for robot learning," *arXiv preprint arXiv:2108.10470*, 2021.

[2] E. Todorov, T. Erez, and Y. Tassa, "Mujoco: A physics engine for model-based control," in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 5026–5033.

[3] M. Mittal, P. Roth, J. Tigue *et al.*, "Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning," *arXiv preprint arXiv:2511.04831*, 2025.

[4] J. Tan, T. Zhang, E. Coumans *et al.*, "Sim-to-real: Learning agile locomotion for quadruped robots," in *Robotics: Science and Systems*, 2018.

[5] J. Hwangbo, J. Lee, A. Dosovitskiy *et al.*, "Learning agile and dynamic motor skills for legged robots," *Science robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[6] J. Lee, J. Hwangbo, L. Wellhausen *et al.*, "Learning quadrupedal locomotion over challenging terrain," *Science robotics*, vol. 5, no. 47, p. eabc5986, 2020.

[7] N. Rudin, D. Hoeller, P. Reist *et al.*, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Conference on robot learning*. PMLR, 2022, pp. 91–100.

[8] G. B. Margolis and P. Agrawal, "Walk these ways: Tuning robot control for generalization with multiplicity of behavior," in *Conference on Robot Learning*. PMLR, 2023, pp. 22–31.

[9] G. Ji, J. Mun, H. Kim *et al.*, "Concurrent training of a control policy and a state estimator for dynamic and robust legged locomotion," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4630–4637, 2022.

[10] S. Zhu, R. Huang, L. Mou *et al.*, "Robust robot walker: Learning agile locomotion over tiny traps," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 15 987–15 993.

[11] R. Huang, S. Zhu, Y. Du *et al.*, "Moe-loco: Mixture of experts for multitask locomotion," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 14 218–14 225.

[12] E. Xiao, Y. Dong, J. Lam *et al.*, "Learning stable bipedal locomotion skills for quadrupedal robots on challenging terrains with automatic fall recovery," *npj Robotics*, vol. 3, no. 1, p. 22, 2025.

[13] V. Atanassov, J. Ding, J. Kober *et al.*, "Curriculum-based reinforcement learning for quadrupedal jumping: A reference-free design," *IEEE Robotics & Automation Magazine*, vol. 32, no. 2, pp. 35–48, 2024.

[14] H. Xue, C. Pan, Z. Yi *et al.*, "Full-order sampling-based mpc for torque-level locomotion control via diffusion-style annealing," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 4974–4981.

[15] Y. Ma, A. Cramariuc, F. Farshidian *et al.*, "Learning coordinated badminton skills for legged manipulators," *Science robotics*, vol. 10, no. 102, p. eadu3922, 2025.

[16] J. Ma, W. Liang, H.-J. Wang *et al.*, "Dreureka: Language model guided sim-to-real transfer." RSS, 2024.

[17] Y. Ji, G. B. Margolis, and P. Agrawal, "Dribblebot: Dynamic legged manipulation in the wild," *arXiv preprint arXiv:2304.01159*, 2023.

[18] J. Cheng, D. Kang, G. Fadini *et al.*, "Rambo: Rl-augmented model-based optimal control for whole-body loco-manipulation," *arXiv e-prints*, pp. arXiv–2504, 2025.